Computer Architecture

Project Step 3: ALU Control Unit

Group "Orange": Chloe MacGregor, Nathaniel Jefferson, Rex Almerico

April 29, 2026

1 Objectives

The goal of this step in the project is to integrate the 4-bit ALU components from Step 2 into a control unit that selects the appropriate operation based on a 4-bit opcode. This creates a complete ALU that can perform arithmetic and logic operations on 4-bit inputs.

2 ALU Control Unit Design

The control unit takes a 4-bit opcode and two 4-bit inputs A and B, and produces an 8-bit output based on the selected operation. It instantiates all the 4-bit modules from Step 2 and uses a case statement to select the output.

2.1 Operation Codes

The following table lists the supported operations and their corresponding opcodes:

- 4'b0000 (0) - ADD : out[4:0] = {Cout, A+B}
- 4'b0001 (1) - SUB : out[4:0] = {Bout, A-B}
- 4'b0010 (2) - MUL : out[7:0] = {upper4, lower4} of A*B
- 4'b0011 (3) - DIV : out[7:0] = {remainder, quotient} of A/B
- 4'b0100 (4) - AND : out[3:0] = A & B
- 4'b0101 (5) - OR : out[3:0] = A — B
- 4'b0110 (6) - XOR : out[3:0] = A ˆ B
- 4'b0111 (7) - SHIFT : out[7:4] = bits shifted out, out[3:0] = shifted result
  B[3]=dir(0=L,1=R), B[2:1]=amount(0-3), B[0]=fill

- 4'b1100 (12) - NAND : out[3:0] = $\tilde{(}$A & B)
- 4'b1101 (13) - NOR : out[3:0] = $\tilde{(}$A — B)
- 4'b1110 (14) - XNOR : out[3:0] = $\tilde{(}$A ˆ B)
- 4'b1111 (15) - NOT : out[7:4] = $\tilde{B}$, out[3:0] = $\tilde{A}$

2.2 Verilog Code

```
`include "../step-2/code/arithmetic/adder4.v"
`include "../step-2/code/arithmetic/divider4.v"
`include "../step-2/code/arithmetic/multiplier4.v"
`include "../step-2/code/arithmetic/subtractor4.v"


`include "../step-2/code/logic/not4.v"

`include "../step-2/code/logic/shifter4.v"


`include "../step-2/code/logic/and4.v"
`include "../step-2/code/logic/nand4.v"

`include "../step-2/code/logic/or4.v"
`include "../step-2/code/logic/nor4.v"

`include "../step-2/code/logic/xor4.v"
`include "../step-2/code/logic/xnor4.v"

module control (
    input [3:0] opcode,
    input [3:0] A,
    input [3:0] B,

    output reg [7:0] out
);

    wire [7:0] addOutput;
    wire [7:0] subOutput;
    wire [7:0] mulOutput;
    wire [7:0] divOutput;

    wire [7:0] notOutput;

    wire [7:0] andOutput;
    wire [7:0] nandOutput;
    wire [7:0] orOutput;
    wire [7:0] norOutput;
    wire [7:0] xorOutput;
    wire [7:0] xnorOutput;
```

```
    wire [7:0] shiftOutput;



    adder4     ADD(A, B, 1'b0, addOutput[3:0], addOutput[4]);
    subtractor4 SUB(A, B, 1'b0, subOutput[3:0], subOutput[4]);
    multiplier4 MUL(A, B, mulOutput[3:0], mulOutput[7:4]);
    divider4   DIV(A, B, divOutput[3:0], divOutput[7:4]);



    not4       NOTA(A, notOutput[3:0]);
    not4       NOTB(B, notOutput[7:4]);

    shifter4   SHIFT(A, B, shiftOutput[3:0], shiftOutput[7:4]);


    and4       AND(A, B, andOutput[3:0]);
    nand4      NAND(A, B, nandOutput[3:0]);
    or4        OR(A, B, orOutput[3:0]);
    nor4       NOR(A, B, norOutput[3:0]);
    xor4       XOR(A, B, xorOutput[3:0]);
    xnor4      XNOR(A, B, xnorOutput[3:0]);


    always @(*) begin
        case (opcode)
            4'b0000: out = addOutput;
            4'b0001: out = subOutput;
            4'b0010: out = mulOutput;
            4'b0011: out = divOutput;

            4'b1111: out = notOutput;
            4'b0111: out = shiftOutput;

            4'b0100: out = andOutput;
            4'b0101: out = orOutput;
            4'b0110: out = xorOutput;

            4'b1100: out = nandOutput;
            4'b1101: out = norOutput;
            4'b1110: out = xnorOutput;
        endcase
    end

endmodule
```

3 Unit Tests

The control unit is tested with a comprehensive testbench that verifies all operations with various inputs.

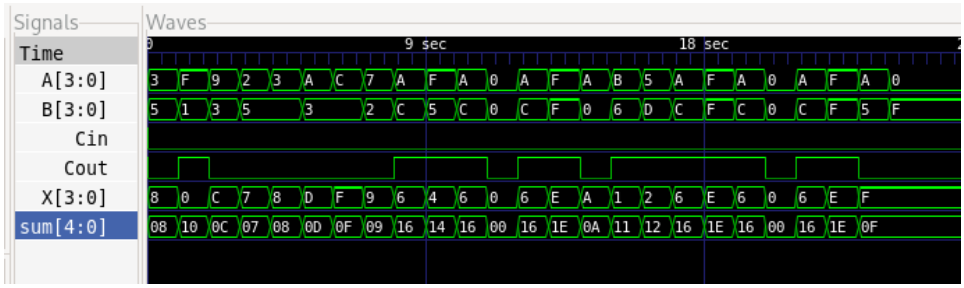
```
// ALU Opcode Table:
//   4'b0000 (0) - ADD  : out[4:0] = {Cout, A+B}
//   4'b0001 (1) - SUB  : out[4:0] = {Bout, A-B}
//   4'b0010 (2) - MUL  : out[7:0] = {upper4, lower4} of A*B
//   4'b0011 (3) - DIV  : out[7:0] = {remainder, quotient} of A/B
//   4'b0100 (4) - AND  : out[3:0] = A & B
//   4'b0101 (5) - OR   : out[3:0] = A | B
//   4'b0110 (6) - XOR  : out[3:0] = A ^ B
//   4'b0111 (7) - SHIFT : out[7:4] = bits shifted out, out[3:0] =
    shifted result
//                        B[3]=dir(0=L,1=R), B[2:1]=amount(0-3),
    B[0]=fill
//   4'b1100 (12) - NAND : out[3:0] = ~(A & B)
//   4'b1101 (13) - NOR : out[3:0] = ~(A | B)
//   4'b1110 (14) - XNOR : out[3:0] = ~(A ^ B)
//   4'b1111 (15) - NOT : out[7:4] = ~B, out[3:0] = ~A

module control_tb();
    reg  [3:0] opcode, A, B;
    wire [7:0] out;
    integer errors;

    control DUT (.opcode(opcode), .A(A), .B(B), .out(out));

    // Full 8-bit check (MUL, DIV, NOT, SHIFT)
    task check8;
        input [7:0] expected;
        begin
            #1;
            if (out !== expected) begin
                errors = errors + 1;
                $display("FAIL control: opcode=%b A=%b B=%b -> out=%b
                    (expected %b)",
                        opcode, A, B, out, expected);
            end else
                $display("PASS control: opcode=%b A=%b B=%b ->
                    out=%b", opcode, A, B, out);
        end
    endtask

    // Lower 5-bit check (ADD, SUB - result in [3:0], carry/borrow
        in [4])
    task check5;
        input [4:0] expected;
        begin
            #1;
            if (out[4:0] !== expected) begin
```

```
            errors = errors + 1;
            $display("FAIL control: opcode=%b A=%b B=%b ->
                out[4:0]=%b (expected %b)",
                    opcode, A, B, out[4:0], expected);
        end else
            $display("PASS control: opcode=%b A=%b B=%b ->
                out[4:0]=%b", opcode, A, B, out[4:0]);
    end
endtask

// Lower 4-bit check (AND, OR, XOR, NAND, NOR, XNOR)
task check4;
    input [3:0] expected;
    begin
        #1;
        if (out[3:0] !== expected) begin
            errors = errors + 1;
            $display("FAIL control: opcode=%b A=%b B=%b ->
                out[3:0]=%b (expected %b)",
                    opcode, A, B, out[3:0], expected);
        end else
            $display("PASS control: opcode=%b A=%b B=%b ->
                out[3:0]=%b", opcode, A, B, out[3:0]);
    end
endtask

initial begin
    errors = 0;
    // --- ADD (4'b0000): out[3:0]=sum, out[4]=Cout ---
    opcode = 4'b0000; A = 4'b0011; B = 4'b0101;
        check5(5'b01000); // 3+5=8
    opcode = 4'b0000; A = 4'b1111; B = 4'b0001;
        check5(5'b10000); // 15+1=16, carry out

    // --- SUB (4'b0001): out[3:0]=diff, out[4]=Bout ---
    opcode = 4'b0001; A = 4'b1001; B = 4'b0011;
        check5(5'b00110); // 9-3=6
    opcode = 4'b0001; A = 4'b0010; B = 4'b0101;
        check5(5'b11101); // 2-5, borrow, result=13

    // --- MUL (4'b0010): out[3:0]=lower4, out[7:4]=upper4 ---
    opcode = 4'b0010; A = 4'b0011; B = 4'b0101;
        check8(8'b00001111); // 3*5=15
    opcode = 4'b0010; A = 4'b1010; B = 4'b0011;
        check8(8'b00011110); // 10*3=30

    // --- DIV (4'b0011): out[3:0]=quotient, out[7:4]=remainder
        ---
    opcode = 4'b0011; A = 4'b1100; B = 4'b0011;
        check8(8'b00000100); // 12/3=4 r0
    opcode = 4'b0011; A = 4'b0111; B = 4'b0010;
        check8(8'b00010011); // 7/2=3 r1
```

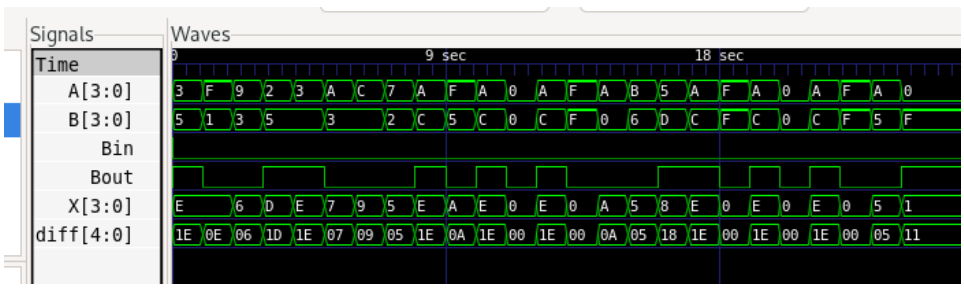
```
        // --- AND (4'b0100) ---
        opcode = 4'b0100; A = 4'b1010; B = 4'b1100; check4(4'b1000);
        opcode = 4'b0100; A = 4'b1111; B = 4'b0101; check4(4'b0101);

        // --- OR (4'b0101) ---
        opcode = 4'b0101; A = 4'b1010; B = 4'b1100; check4(4'b1110);
        opcode = 4'b0101; A = 4'b0000; B = 4'b0000; check4(4'b0000);

        // --- XOR (4'b0110) ---
        opcode = 4'b0110; A = 4'b1010; B = 4'b1100; check4(4'b0110);
        opcode = 4'b0110; A = 4'b1111; B = 4'b1111; check4(4'b0000);

        // --- SHIFT (4'b0111): out[3:0]=shifted_result,
            out[7:4]=shifted_out ---
        opcode = 4'b0111; A = 4'b1010; B = 4'b0000;
            check8(8'b00001010); // no shift
        opcode = 4'b0111; A = 4'b1011; B = 4'b0110;
            check8(8'b01011000); // left 3, fill 0
        opcode = 4'b0111; A = 4'b0101; B = 4'b1101;
            check8(8'b01001101); // right 2, fill 1

        // --- NAND (4'b1100) ---
        opcode = 4'b1100; A = 4'b1010; B = 4'b1100; check4(4'b0111);
        opcode = 4'b1100; A = 4'b1111; B = 4'b1111; check4(4'b0000);

        // --- NOR (4'b1101) ---
        opcode = 4'b1101; A = 4'b1010; B = 4'b1100; check4(4'b0001);
        opcode = 4'b1101; A = 4'b0000; B = 4'b0000; check4(4'b1111);

        // --- XNOR (4'b1110) ---
        opcode = 4'b1110; A = 4'b1010; B = 4'b1100; check4(4'b1001);
        opcode = 4'b1110; A = 4'b1111; B = 4'b1111; check4(4'b1111);

        // --- NOT (4'b1111): out[3:0]=~A, out[7:4]=~B ---
        opcode = 4'b1111; A = 4'b1010; B = 4'b0101;
            check8(8'b10100101); // ~A=0101, ~B=1010
        opcode = 4'b1111; A = 4'b0000; B = 4'b1111;
            check8(8'b00001111); // ~A=1111, ~B=0000

        if (errors == 0)
            $display("All control tests passed.");
        else
            $fatal(1, "%0d control test(s) failed.", errors);
        #5 $finish;
    end

    initial begin
        $dumpfile("waveform_control.vcd");
        $dumpvars(0, control_tb);
    end
endmodule
```

4 Waveforms for Each Operation

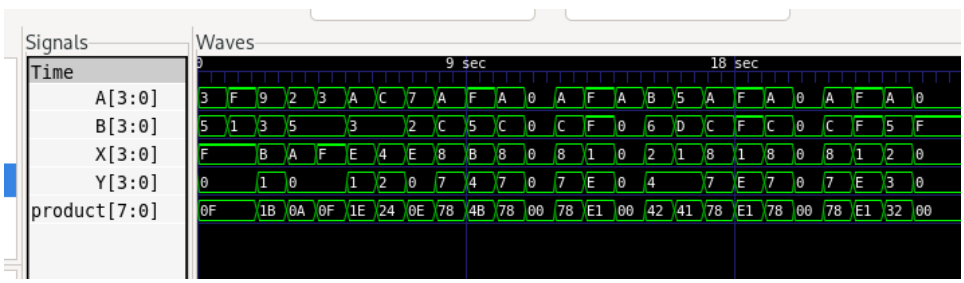
4.1 ADD (Opcode 0000)



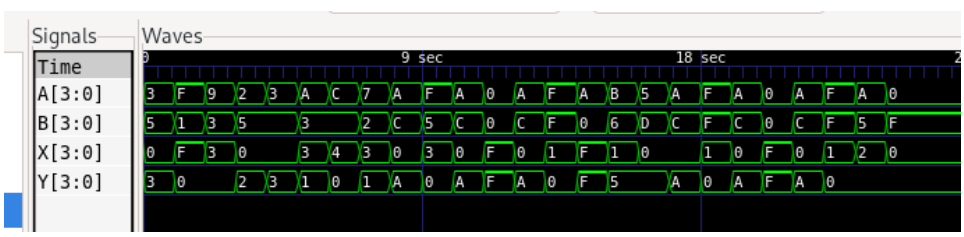
4.2 SUB (Opcode 0001)



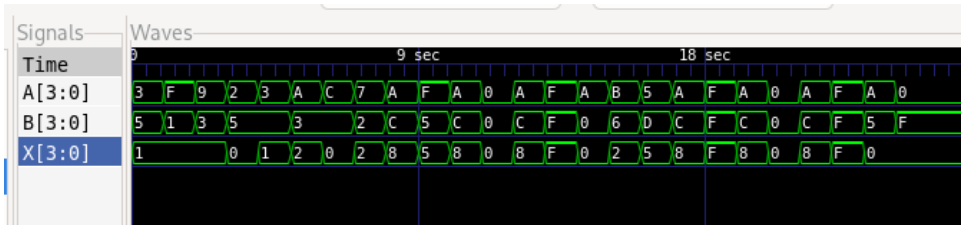
4.3 MUL (Opcode 0010)



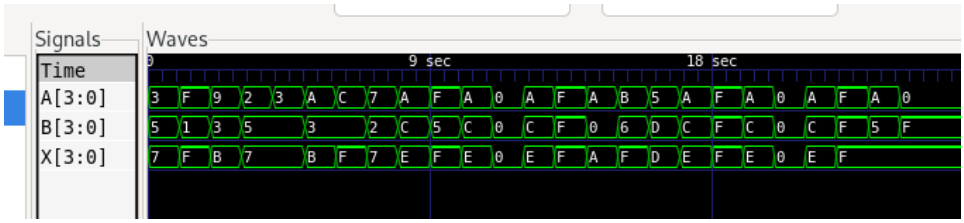
4.4 DIV (Opcode 0011)



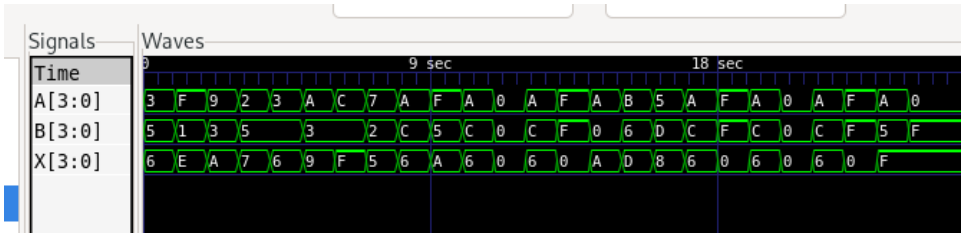
4.5 AND (Opcode 0100)



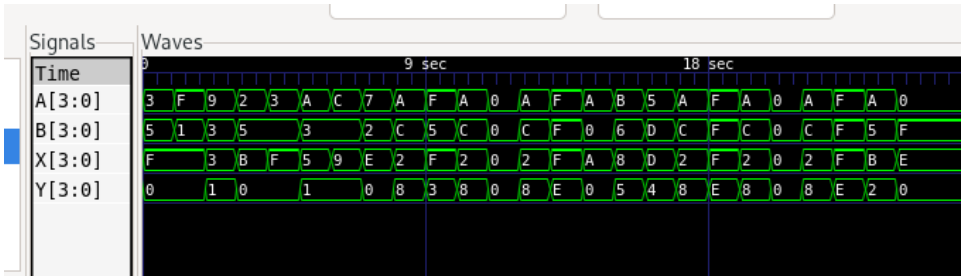
4.6 OR (Opcode 0101)



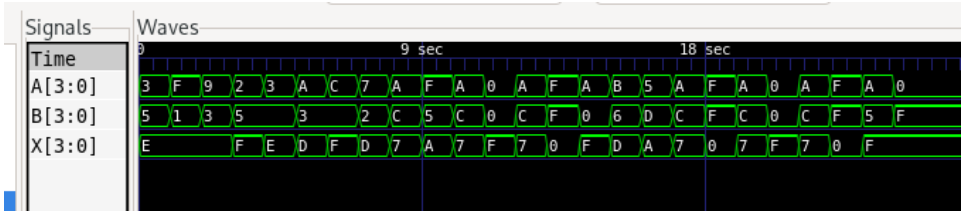
4.7 XOR (Opcode 0110)



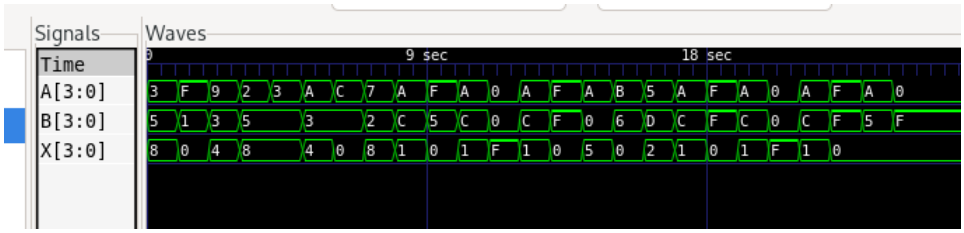
4.8 SHIFT (Opcode 0111)



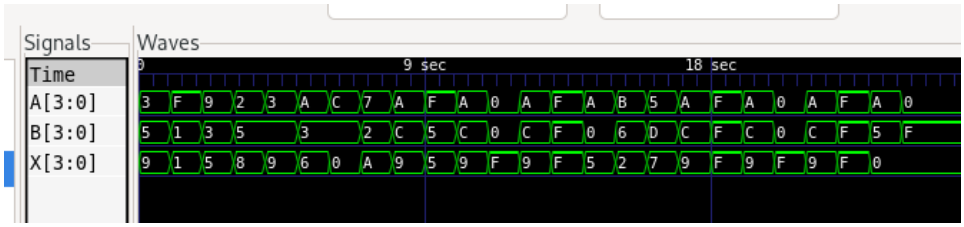
4.9 NAND (Opcode 1100)



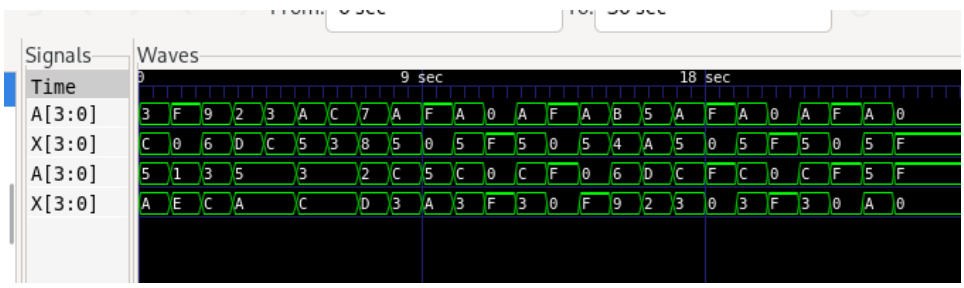
4.10 NOR (Opcode 1101)



4.11 XNOR (Opcode 1110)



4.12 NOT (Opcode 1111)



5 Conclusion

In this step of the project, we successfully integrated the individual 4-bit ALU components into a unified control unit. The control unit uses opcodes to select between different arithmetic and logic operations, producing the correct 8-bit output. This completes the ALU design for this computer architecture project.